

trpc-go服务天机阁接入方法（极简傻瓜教程）

本教程仅限于trpc-go服务，啥也不用管，照着做，照着测就完事儿了。

详细教程可参考http://km.oa.com/articles/show/483131?kmref=search&from_page=1&no=4

一、接入效果

<http://tdocs.tpstelemetry.oa.com/?orgId=1>

业务看板：

服务监控：

调用链路：

二、租户申请

参考<http://tpstelemetry.pages.oa.com/02.guide.html>。如果你的团队已经申请了，你可以直接看下一步。

三、改代码

Step 1. 配置trpc_go.yaml

红框中为天机阁相关配置。文档的同学可以直接copy，其他部门可以根据申请的租户进行调整。

需要注意的是，fraction: 1代表上报采样率为100%，可根据需求调整为大于0小于1的任意数值。

Step 2. 修改server.go (或者main.go ?)

加几行代码：

```

1 // 匿名引入天机阁tRPC拦截器
2 _ "git.code.oa.com/tpstelemetry/tps-sdk-go/instrumentation/trpctelemetry"
3
4 // 实现traces.DefaultTraceAttributesFunc方法，配置染色key
5 func init() {
6     traces.DefaultTraceAttributesFunc = func(ctx context.Context, req interface{})
7     []label.KeyValue {
8         if newReq, ok := req.(api.Req); ok {
9             if newReq.GetHeader() != nil && newReq.GetHeader().GetDocToken()
10             != nil {
11                 return
12                 []label.KeyValue{label.Key("uid").String(newReq.GetHeader().GetDocToken().GetTiny
13                 Id())}
14             }
15         }
16     }
17 }

```

稍微讲解一下，traces.DefaultTraceAttributesFunc是实现染色规则的函数。比如我的代码，就是在req的header中找到tinyID字段，并作为uid上报。在后续需要时，可以通过染色的方式将被染色的uid进行的操作trace全量上报。染色网址：

<http://console.tpstelemetry.oa.com/#!/dyeing?id=tdocs>

Step 4. 设置天机阁上报tag

经过以上几步，天机阁的最基本配置就已经完成了，如果你不满足于天机阁上报的有限几个字段，你还可以手动上报你需要的字段，这样某种程度上可以把天机阁当做logging日志系统。我上报的是下图红框里那几个：

在你需要上报的地方，加上以下几行代码：

```
1 // span是在天机阁filter自动加到context中的，不用担心它不存在！唯一需要注意的是你  
   必须使用request中的context  
2 span := trace.SpanFromContext(r.Context())  
3 span.SetAttributes(label.KeyValue{  
4     Key:   "keyName",  
5     Value: "someValue",  
6 })
```

四、测试

Step 1. 部署和访问你的服务

不多说

Step 2. 查看上报情况

大概三分钟后（这样比较稳），搜索你在trpc_go.yaml中配置的service.name，搜得到就说明上报成功了。

一点小经验：上报的任何一个字段都可以用来搜，但是貌似不支持模糊搜索。

如果你搜traceID，那么就可以搜到该调用链上的所有记录。

点开任意一条记录，再点击【trace-detail】，再把页面拖到最上面（小坑，一开始我还以为出bug了），就可以看调用链啦~